

Visual Project Manager

Il vostro software dovrà gestire dei Task (compiti), ognuno caratterizzato da un **Titolo**, uno **Stato** (Todo/Doing/Done), delle **Date** (Inizio e Fine) e un **Colore** distintivo.

L'applicazione finale dovrà offrire due visualizzazioni sincronizzate di questi dati:

A. La Dashboard Operativa: Una panoramica immediata per la gestione quotidiana. Ogni task è una "card". Il colore assegnato al task deve essere visibile a colpo d'occhio (come bordo laterale o indicatore) per identificarne la priorità. Da qui si possono creare, modificare ed eliminare i task.

B. Il Diagramma di Gantt: Questa è la sfida principale. È un grafico dove l'asse orizzontale rappresenta il tempo (un calendario). I task non sono più semplici righe di testo, ma **barre colorate** posizionate sulla linea temporale.

- La barra inizia nella data di inizio task.
- La lunghezza della barra rappresenta la durata del task.
- Il colore della barra è lo stesso definito nella Dashboard.

Stack Tecnologico e Vincoli:

Dovete realizzare una Web Application Full-Stack rispettando questi vincoli:

- **Database:** MySQL ospitato in Cloud su **Aiven**.
- **Backend:** Python **Flask**.
 - **VINCOLO:** È vietato usare ORM (come SQLAlchemy). Dovete creare una classe Python (DatabaseWrapper.py) che gestisca la connessione usando la libreria **pymysql** e scrivere a mano le query SQL (CREATE TABLE, SELECT, INSERT...).
- **Frontend:** Angular
- **Architettura:** Il Frontend e il Backend devono comunicare esclusivamente tramite API REST JSON.

L'esercitazione è strutturata in un percorso incrementale, **NON potete saltare gli step**. Per superare ogni step, dovete completare la funzionalità richiesta e fare un git commit + git push.

Sarà valutato il vostro lavoro ripercorrendo la cronologia dei commit.

COMMIT 1: Inizializzazione del Progetto

- Creare la repository GitHub
- Creare la struttura delle cartelle principale (/backend e /frontend)
- **Backend:** Inizializzare l'ambiente Python, creare il file requirements.txt con le dipendenze iniziali (Flask, pymysql, flask-cors, python-dotenv) e installarle
- **Frontend:** Generare il nuovo progetto Angular con la CLI (ng new frontend)
- Aggiungere un .gitignore configurato correttamente per escludere file di configurazione sensibili

Messaggio Commit: 01 - Project Init

COMMIT 2: Il "Wrapper" del Database

- Recuperare le credenziali da Aiven
- Creare la classe DatabaseWrapper.py nel backend e implementare il metodo di connessione
- Implementare le query (CREATE TABLE...) necessarie per supportare le funzionalità richieste (possono essere anche aggiunte in seguito)

Messaggio Commit: 02 - Database Wrapper

COMMIT 3: Backend API

- Configurare l'app Flask principale e abilitare **CORS**.
- Implementare l'endpoint POST /tasks: riceve un JSON dal frontend, lo valida (assicurarsi che ci siano date e colore) e lo inserisce nel DB usando il Wrapper.
- Implementare l'endpoint GET /tasks: legge tutti i task dal DB e li restituisce come lista JSON.

Messaggio Commit: 03 - CRUD API

COMMIT 4: Frontend Dashboard

- Creare il TaskService in Angular per dialogare con il backend.
- Creare il componente per la Dashboard (/tasks).
- Mostrare la lista dei task recuperati dall'API.
- **Requisito UI:** Ogni task deve essere una "card" che mostra i dati principali. Non basta stampare il testo: l'interfaccia deve essere pulita.

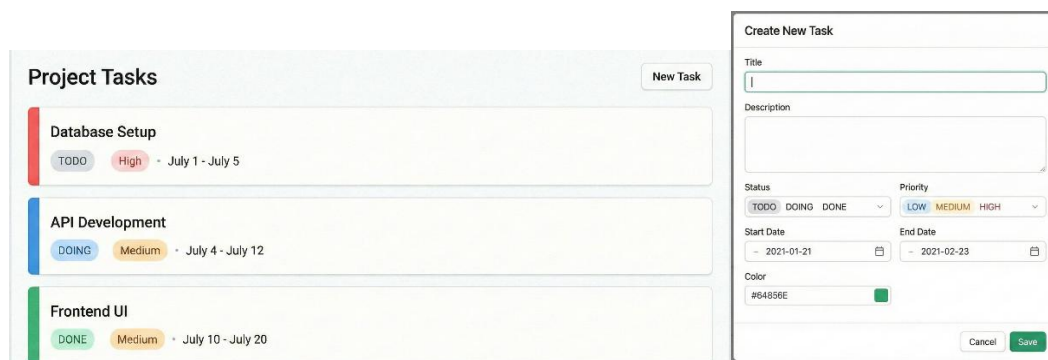
(Inserendo manualmente dei dati nel DB, aprendo la pagina Angular questi appaiono listati correttamente)



Messaggio Commit: 04 - Frontend Read-Only Dashboard

COMMIT 5: Integrazione Colori e Form di Creazione

- Creare un componente Form (pulsante "New Task") per l'inserimento.
- Il form **deve** includere: input di testo, selettori di date e un **input tipo "color"** (<input type="color">) per scegliere il colore HEX.
- Collegare il form all'API POST creata prima.
- Aggiornare la Dashboard: le card devono ora usare il colore salvato nel DB per decorarsi visivamente (es. un bordo laterale colorato, un'icona o lo sfondo del badge di stato).



Messaggio Commit: 05 - Task Creation and Color Integration

COMMIT 6: Il Diagramma di Gantt

- Creare il componente per il diagramma di Gantt (/gantt).
- **Logica:** Implementare (nel frontend o backend) il calcolo della **durata** in giorni di ogni task e del suo **offset** (ritardo) rispetto alla data di inizio del primo task del progetto.
- **Grafica:** Renderizzare una griglia temporale (es. una tabella HTML dove ogni colonna è un giorno).
- Usare i valori calcolati per disegnare delle barre orizzontali colorate che occupano le celle corrette della griglia.



Messaggio Commit: 06 - Gantt Chart Implementation

COMMIT 7: Rifiniture e Funzionalità Complete

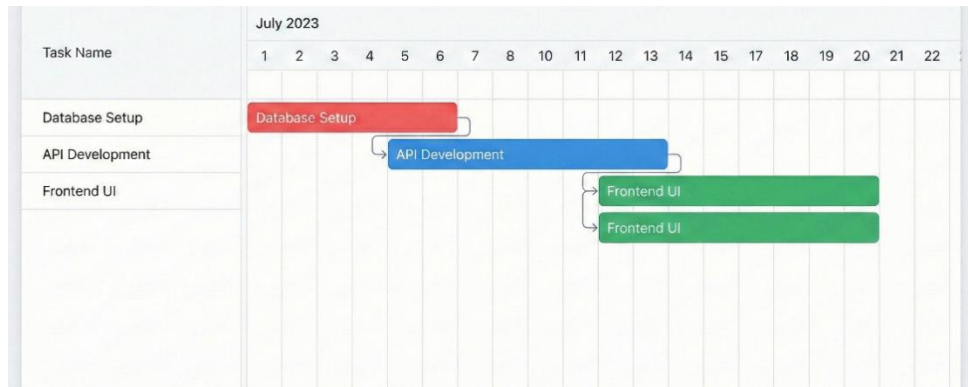
- Implementare nel backend le API mancanti: PUT /tasks/<id> (modifica) e DELETE /tasks/<id>.
- Aggiungere i tasti "Edit" e "Delete" sulle card della Dashboard e collegarli.
- Assicursi che modificando un colore o una data nella Dashboard, il cambiamento si rifletta immediatamente anche nel Diagramma di Gantt.

Messaggio Commit: 07 - Feature Complete

COMMIT 8: Gestione Dipendenze tra Task

- Aggiungere la creazione di una **tabella** task_dependencies, questa tabella deve avere due colonne: predecessor_id e successor_id, entrambe chiavi esterne che puntano all'id della tabella tasks
- Creare un endpoint POST /tasks/<id>/dependency: Riceve l'ID del task predecessore e crea il legame nel DB.
- Creare un endpoint DELETE /tasks/<id>/dependency: Rimuove il legame.
- Aggiornare la GET /tasks (o creare una GET /tasks/<id>) in modo che restituisca anche l'informazione se un task è bloccato da un altro.
- Nel **Form di Creazione/Modifica**, aggiungere un menu a tendina (dropdown) chiamato "Dipende da..." che elenca gli altri task esistenti. Selezionandone uno, si crea la dipendenza.
- Nella **Dashboard**, se un task ha una dipendenza non risolta, mostrare un piccolo avviso visivo (es. un'icona di lucchetto o una scritta "Bloccato da: [Nome Task]").

(Se provo a segnare il Task B dipendente da A come "DONE" mentre il Task A è ancora "TODO", il sistema mi dà un errore)



Messaggio Commit: 08 - Task Dependencies Logic